THE FRAMEWORK | START HERE

One game. Three connected systems.

The lesson moves an existing Snake game into a Vite project. Understand the responsibilities first; then every import, class, and callback has a place to belong.

1 / PREPARE

Development tools

Node runs Vite. npm installs tools and runs commands. Vite serves or builds the project.

2 / CONNECT

Browser resources

HTML starts the app. Modules connect JavaScript, CSS, images, and audio. Resources must be ready when used.

3 / RUN

Game behavior

Input changes direction. Rules change state. Canvas displays that state. A timer repeats the process.

The game inside the project

Input
arrow key

→

State + rules
coordinates, collisions

→

Output
pixels + sound

The timer creates the next turn through this chain. **State** is what the program remembers: the snake's cells, the apple's cell, direction, score, delay, and whether play has ended. The canvas is a picture of that state.

> **The central idea**
> Vite changes how the game is prepared and how its files are connected. The JavaScript game rules still determine how Snake behaves.

How to read this guide

First place a concept in one of the three systems. Next ask what problem it solves. Finally read the code and trace a small example. The detailed pages always return to this framework.

Route: project map (2) • tools (3) • resources (4-5) • state and board (6-7) • rules (8-10) • repeated execution (11) • pitfalls (12) • practice (13) • reference (14).

What changed, and where it lives

The earlier game is the conceptual anchor. Most of its movement, scoring, drawing, and keyboard logic is carried into src/main.js.

Responsibility	Earlier files in js/74	This Vite project
Start the page	snake.html	index.html loads src/main.js as a module.
Style the page	Inline style block	src/style.css, imported by JavaScript.
Find pictures	Literal image paths	PNG imports produce URLs assigned to img.src.
Create sounds	Audio elements in HTML	MP3 imports plus createElement('audio').
Run the game	snake.js	src/main.js retains the game rules.
Prepare and build	No Vite package setup in those files	package.json, lockfile, and npm scripts.

Read the folders as responsibilities

```
usingVite/
  index.html          # page shell and module entry
  package.json        # commands and dependency requirements
  package-lock.json   # resolved dependency versions
  .gitignore          # files Git should ignore
  src/
    main.js           # resource setup and game code
    style.css         # page and canvas presentation
    images/           # snakeHead.png and apple.png
    media/            # crash.mp3 and crunch.mp3
  public/             # favicon.svg and icons.svg
```

The two SVG files are not referenced by the inspected game source. They are template assets, not part of the Snake rules. No custom Vite configuration file is present.

What is actually new here?

Module scripts were already used in the earlier HTML. The main additions are the Vite workflow, CSS and asset imports, the project structure, and creating audio in JavaScript. Classes and canvas are supporting concepts, explained because they make this project understandable.

Source: index.html; src/main.js; src/style.css; package.json; .gitignore; folder inventory; ../snake.html and ../snake.js.

Tools prepare the program; the browser plays it

Node, npm, and Vite solve different problems. Keeping their jobs separate makes the commands easier to remember.

Tool or file	Job in this project
Node.js	Runs development tools on your computer. The game itself uses browser APIs such as document and canvas.
npm	Installs packages and runs the named scripts in package.json.
Vite	Provides the development server and produces browser-ready build files.
package.json / lockfile	Requested dependencies and commands / exact resolved versions, including Vite 8.3.6.
node_modules / dist	Installed tools / generated build output. Both are ignored by Git here.

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
}
```

Development: edit source and open Vite's printed URL. **Build:** prepare deployable files in `dist`. **Preview:** locally inspect that build; it is not a production hosting service. Saving CSS can update styles through hot module replacement (HMR); JavaScript edits in this app may reload and reset the game.

Commands for this existing project

```
cd C:\xampp\htdocs\Class\class\js\74\usingVite
npm ci
npm run dev
# Later, stop dev with Ctrl+C if desired:
npm run build
npm run preview
```

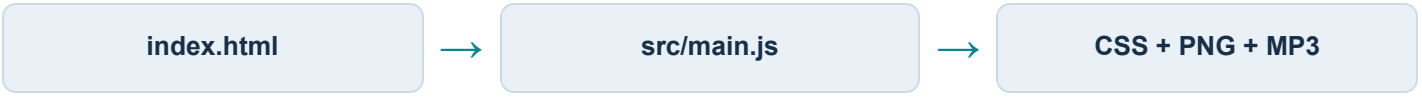
`npm ci` installs from the lockfile and replaces an existing node_modules folder. Use the URL printed by Vite; opening index.html directly, or serving raw source through XAMPP alone, does not perform Vite's transformations.

`"private": true` blocks accidental npm publication; it does not make the website private. `devDependencies` contains build tools. `^8.3.6` allows compatible releases below 9.0.0; the lockfile pins the current resolution. The locked Vite package declares Node `^20.19.0 || >=22.12.0`.

Source: package.json and package-lock.json. References: Vite Getting Started; npm package.json (links on page 14). Commands are instructions, not a record of a game build performed for this guide.

Imports declare what the program needs

Think of the project as a dependency graph: an entry file points to the resources it needs, and Vite follows those connections.




```
<script src="src/main.js" type="module"></script>

import './style.css';
import snakeSrc from './images/snakeHead.png';
import crashSoundSrc from './media/crash.mp3';
```

Two import shapes, two jobs

The CSS import has a side effect. It does not bind a variable. Its purpose is to apply styling. Vite handles this CSS import and its development updates.

The asset imports give you URLs. `snakeSrc` is a string locating the image, not an image element and not the picture's pixels. The name is chosen by the programmer. The browser still needs an image object to load and draw it.

```
const snakeHead = document.createElement('img');
snakeHead.src = snakeSrc;
```

`./` means relative to the importing file. Because main.js is in src, `./images/snakeHead.png` resolves inside src/images. Vite can rewrite an imported URL for the build, often using a hashed filename or inlining a small asset.

Two meanings of module in the setup

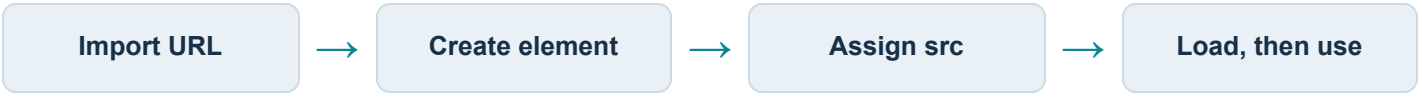
`type="module"` on the script tells the **browser** to treat JavaScript as a module. It runs after HTML parsing and keeps top-level bindings out of the ordinary global scope. `"type": "module"` in package.json tells **Node** how to interpret .js files in that package. Neither setting alone teaches a plain browser to import these PNG, MP3, or CSS files as written; Vite supplies that handling.

> **src versus public**
>
> Imported src assets are connected to the build graph. Files under public are copied as-is; public/favicon.svg is referenced as /favicon.svg at the site root, not /public/favicon.svg. This game does not currently use either public SVG.

Source: index.html module script; src/main.js lines 1-11 and 170-179; package.json. References: Vite Assets and Features; MDN Modules.

A resource address is not a ready resource

Creating an element, assigning its URL, and waiting for the data are different steps. This explains why startup uses a callback.




```
const snakeHead = document.createElement('img');
snakeHead.src = snakeSrc;
snakeHead.onload = () => {
  setTimeout(gameLoop, speed);
};
```

`onload` stores a function for the browser to call later. `() => { ... }` is an arrow function. The game loop is scheduled after the head image loads, with the initial 500 ms delay. Defining a function does not execute its body immediately.

Why initialization order works here

`new Snake()` runs before `new Apple()`. That matters because the Apple constructor calls move(), which reads the snake's segments. Drawing methods mention image variables declared farther down, but those methods are not called until the loop starts later.

The code assigns the apple image's src too, but only waits for the **snake head** image. Waiting 500 ms is not a guarantee that the apple image has loaded. A more robust startup would register handlers before setting src, wait for both images, and handle loading errors. That is a proposed improvement, not what the current code does.

Audio follows the same resource model

```
import crunchSoundSrc from './media/crunch.mp3';
const crunch = document.createElement('audio');
crunch.src = crunchSoundSrc;

// When the snake eats:
crunch.currentTime = 0;
crunch.play();
```

The audio element can play without being appended to the visible page. Setting `currentTime` to zero restarts the sound from the beginning. A crash uses a separate audio object. The old HTML audio elements and querySelector calls are commented out.

> **Browser policy is another readiness condition**
>
> play() returns a Promise, representing a result that arrives later. It can reject when autoplay is blocked or playback fails. The current code does not handle rejection. A Start button and explicit error handling would make sound behavior more predictable.

Source: src/main.js lines 5-11, 147-150, 167-179; index.html audio comments. Reference: MDN HTMLMediaElement.play().

The game is data before it is a picture

Classes group related state and behavior. They let the loop ask a snake to move and an apple to relocate without repeating those details.

State	Meaning
snake.segments	Ordered array of { x, y } objects. Index 0 is the head.
apple.x / apple.y	Current apple position on the grid.
direction	Requested movement, initially ArrowRight.
score / speed / gameOver	Apples eaten / delay in milliseconds / stop flag.

```
class Snake {
  segments = [{ x: -SNAKE_SIZE, y: 0 }];
  get head() {
    return this.segments[0];
  }
}
const snake = new Snake();
```

Class: the definition of a kind of object. **Instance:** the particular object created with `new`. **Field:** data stored on that instance. The field initializer creates a segments array for each Snake instance.

this: the receiver of a method call. In `snake.move()`, this refers to snake. **Getter:** a method read like a property: `snake.head`, not `snake.head()`. It returns the actual first segment object, not a copy.

```
class Apple {
  constructor() { this.move(); }
  static getRandomNumber(max) {
    return Math.floor(Math.random() * (max + 1));
  }
}
```

The **constructor** runs during `new Apple()` and chooses an initial spot. A **static method** belongs to the class: `Apple.getRandomNumber(...)`. It is a helper that does not require a particular apple's stored coordinates.

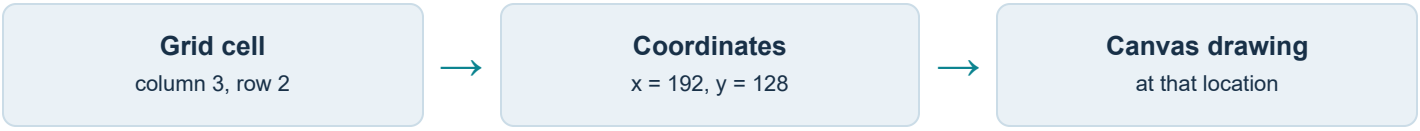
Scope and mutation

const prevents reassigning a binding; it does not freeze an object. A const snake can still change its segments. direction, score, speed, and gameOver use let because their values are reassigned. These shared variables live in the module scope, so the classes depend on this surrounding game setup.

Source: src/main.js lines 17-43, 107-137, and 167-168. Class excerpts omit other members to focus on these concepts.

The grid connects coordinates to pixels

The rules treat the board as 64-pixel cells. Canvas turns those stored coordinates into a visible frame.



The origin `(0, 0)` is the top-left corner. x increases rightward; y increases downward. A move changes one coordinate by `SNAKE_SIZE`, which is 64.

```
theCanvas.width = window.innerWidth
  - (window.innerWidth % SNAKE_SIZE);
```

`%` gives the remainder. For a 1,000-pixel window, `1000 % 64` is 40, so the canvas drawing width becomes 960: exactly 15 cells. The last legal cell starts at `960 - 64 = 896`. Height follows the same rule.

Two separate kinds of sizing

`canvas.width` and `canvas.height` set the drawing surface. CSS controls the page layout and displayed box. Setting the canvas dimensions also clears the image and resets drawing state.

The CSS removes the body's default margin, makes the canvas a block element, and adds a 1-pixel black border. `box-sizing: border-box` controls how CSS box dimensions account for borders and padding; it does not change the 64-unit game coordinates.

Drawing call	What it means
getContext('2d')	Obtain the browser drawing interface.
clearRect(...)	Erase the previous frame.
drawImage(img, x, y)	Paint an image at its natural size in this code.
fillStyle + fillRect(...)	Set a color, then paint a solid body cell.
font + measureText + fillText	Measure and paint the score text.

`beginPath()` starts a new path, but `fillRect()` paints directly and does not need it. The extra call in the body loop is harmless.

> **Resizing changes the board, not the stored positions**
>
> The resize listener recalculates canvas dimensions. It does not relocate a snake or apple left outside the new bounds. On a window smaller than one cell, a dimension can become zero.

Source: src/main.js lines 13-24, 96-103, 157-161; src/style.css.

Movement rearranges the state

The snake moves by calculating a candidate head position, checking it, and recycling its old tail object as the new head.

Step 1: propose a move

```
let tempX = this.head.x;
let tempY = this.head.y;
// In the ArrowRight case:
tempX += SNAKE_SIZE;
```

The temporary values keep the current state intact while rules check the candidate. `switch` chooses the matching direction; each `break` stops that case from falling into the next one.

Step 2: commit a valid move

```
const oldTail = this.segments.pop();
oldTail.x = tempX;
oldTail.y = tempY;
this.segments.unshift(oldTail);
```

`pop()` removes and returns the last object. After changing that object's coordinates, `unshift()` inserts it at the front. The remaining objects stay at their previous positions and become the following body cells.

Example: all y values are 0; travel is rightward

Before: head 128 | body 64 | tail 0

Candidate: **128 + 64 = 192**

After: head 192 | body 128 | tail 64

Step 3: growth increases the stored length

```
this.segments.push({
  x: this.segments.at(-1).x,
  y: this.segments.at(-1).y
});
```

`at(-1)` reads the last item. `push()` appends a new object with the same coordinates as the tail. The two tail entries briefly overlap; on the next move, recycling the extra entry creates a longer visible body. Copying coordinates into a new object avoids making both entries refer to the same mutable object.

> **The initial -64 is deliberate**
>
> The snake starts one cell left of the board. Its first default rightward move places the head at (0, 0). A different direction chosen before that first move can cause an immediate boundary collision.

Source: src/main.js lines 35-93 and 145-154. Worked positions are illustrative, not a captured play session.

Input requests a direction; rules permit a move

Keyboard events and timer ticks are separate. A key changes direction now; the next game tick uses the direction then stored.

```
const opposites = {
  ArrowUp: 'ArrowDown', ArrowDown: 'ArrowUp',
  ArrowLeft: 'ArrowRight', ArrowRight: 'ArrowLeft'
};
// Inside the arrow-key cases:
if (isOnPiece || e.key !== opposites[direction])
  direction = e.key;
```

`e` is the keyboard event; `e.key` identifies the key. Bracket notation looks up a property using the current value of direction. If direction is ArrowRight, `opposites[direction]` is ArrowLeft.

`isOnPiece` is the code's name for "the snake has one segment." `||` means OR: allow any turn for one segment, or allow a direction that is not the opposite. `!==` means strict inequality. Several case labels share one handler.

Boundary collision: reject a cell outside the board

```
tempX < 0 || tempX > theCanvas.width - SNAKE_SIZE ||
tempY < 0 || tempY > theCanvas.height - SNAKE_SIZE
```

The far edge is width minus one cell because a segment must fit completely. On collision, the code sets `gameOver = true` and plays the crash sound. It skips committing the candidate and draws the snake at its existing position.

Self-collision: compare with occupied cells

The loop checks segment indexes **3 through length - 2**. Each comparison needs both x and y to match, so it uses `&&` (AND). It excludes the last tail entry, which ordinarily leaves its cell during movement. Skipping early segments relies on assumptions about allowed turns and the snake's shape.

> **A timing hole in the current rules**
>
> While moving right with multiple segments, pressing Up and then Left before the next tick passes both immediate checks. The next move can therefore reverse into the neck. The collision loop skips that neck. A stronger design accepts at most one turn per tick or validates against the last direction actually moved.

The commented older expression `!direction === 'ArrowDown'` is incorrect: ! converts the string to a boolean before comparison. The active lookup-based check avoids that expression.

Source: src/main.js lines 56-91 and 181-226. The rapid-key scenario follows from the code; it was not reproduced in a browser session.

Apple placement combines randomness and a rule

Randomness proposes a location. The occupancy rule decides whether to accept it. This repeats until a candidate is valid.

```
while (!valid) {
  const x = Apple.getRandomNumber(theCanvas.width);
  this.x = x - (x % SNAKE_SIZE);
  // The same calculation sets this.y.
  if (!snake.segments.some(
    s => s.x === this.x && s.y === this.y
  )) {
    valid = true;
  }
}
```

`Math.random()` returns a value from 0 up to, but not including, 1. `Math.floor()` rounds down. The helper's `max + 1` makes the integer range include max. Subtracting the remainder snaps a coordinate to the start of a cell: 173 becomes 128.

Read the condition as an English sentence

`some(...)` asks, "Does at least one segment occupy this candidate?" The callback receives one segment as `s` and returns true when both coordinates match. `some` stops at the first match. The outer `!` negates its result: accept the spot when **no** segment occupies it.

The arrow callback uses the surrounding `this` from Apple.move(), so `this.x` and `this.y` still mean the apple's proposed coordinates.

> **A real boundary bug**
>
> With a width of 960, the current helper can return 960. Snapping leaves it at 960, but the final legal x is 896. An apple can therefore be placed entirely off the board. Checking snake occupancy alone does not check board bounds.

Proposed improvement: choose a cell number directly

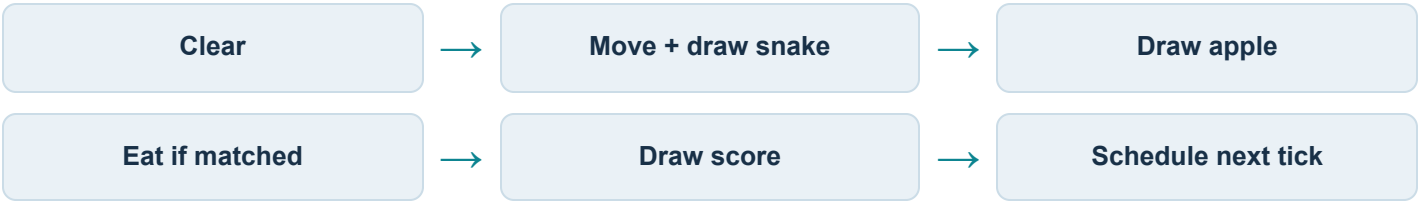
```
const columns = theCanvas.width / SNAKE_SIZE;
const rows = theCanvas.height / SNAKE_SIZE;
this.x = Math.floor(Math.random() * columns) * SNAKE_SIZE;
this.y = Math.floor(Math.random() * rows) * SNAKE_SIZE;
```

This replacement excludes the far edge when the board has positive dimensions. Keep the occupancy check. A complete version must also handle a zero-size board and a full board, where repeatedly searching for an empty legal cell cannot succeed. Choosing from a list of free cells makes that case explicit.

Source: src/main.js lines 107-137. The first excerpt omits y calculation and logs; the replacement is a teaching example. Project files have not been changed.

The loop coordinates the whole game

gameLoop is the place where state, rules, drawing, and time meet. Its order explains what you see after each tick.



Trace a tick that eats an apple

Phase	What happens in this implementation
1. Guard	If gameOver is already true, the function does nothing.
2. Move	snake.move() proposes and checks a position, commits it if valid, then draws.
3. Apple	apple.draw() paints the current apple position.
4. Eat	Matching head/apple coordinates increase score, shorten delay, play crunch, grow, and relocate the apple.
5. Score	A template literal creates the label, then the text is measured and drawn.
6. Repeat	setTimeout schedules one future call using the current delay.

```
score++;
speed *= .95;
// ...
const scoreText = `Score: ${score}`;
setTimeout(gameLoop, speed);
```

`score++` adds one. `speed *= .95` means `speed = speed * 0.95`. Here "speed" stores a **delay**: 500 ms becomes 475 ms, then 451.25 ms. Shorter waits produce more movement ticks per second. There is no minimum delay in the code.

`setTimeout(gameLoop, speed)` passes the function for later; `gameLoop()` would call it now. The delayed callback does not grow a chain of synchronous call-stack frames. The delay is not an exact clock: browser scheduling and time spent in each tick also affect timing.

Drawing order has visible consequences

The apple and snake are drawn before growth and apple relocation. Their new state appears on the next tick. The score updates in the current tick. If snake.move() sets gameOver, the already-running gameLoop still finishes; the next scheduled call stops at the guard.

The score's x coordinate is canvas.width - measured text width - 10, leaving a 10-unit right gap. The y value 36 is a text baseline. A template literal uses backticks and inserts the value inside `${...}` into the string.

Source: src/main.js lines 139-175, especially gameLoop().

Use the model to explain surprising behavior

Each symptom belongs to a responsibility. Find that responsibility before changing code.

Symptom	Where to look	Reason in this project
Imports fail when opening HTML directly	Prepare / connect	Raw CSS and media imports need Vite processing. Start Vite and use its URL.
An apple never appears on the board	Rules	The inclusive random maximum can produce x = width or y = height. Choose legal cell indexes.
The apple picture is initially missing	Resource readiness	Only the head image is awaited. Wait for both images and handle failures.
Sound does not play	Resource readiness	Playback may be blocked or rejected. Handle play() and use an explicit start interaction.
Fast key presses allow reversal	Input + time	direction can change several times between ticks. Validate a pending turn against the committed direction.
Resizing makes items disappear	Board + state	Dimensions change but positions are not repaired. Decide whether resize resets or remaps the game.
Growth looks one tick late	Output order	Drawing precedes growth and apple relocation. Separate state updates from rendering if changing this.

A cleaner design to aim for later

Read accepted input → **Update all state** → **Render all state** → **Schedule**

This is a conceptual refactoring target, not a description of the current source. It makes one frame represent one coherent game state. Classes can still own movement and apple placement, while a render step owns drawing.

> **Know the limits of this version**
>
> There is no restart screen, explicit win condition, full-board handling, or minimum delay. Arrow-key handling does not call preventDefault(), so browser scrolling may still occur when the page overflows. Debug console logs remain in apple placement and for unhandled keys.

Review scope: this guide is based on reading the complete text source, comparing the earlier version, and checking official documentation. These are source-level findings. The game was not installed, built, or interactively tested as part of this documentation task.

Explain the framework, then predict the details

Answer in words before calculating. The goal is to see why a line exists, not merely remember its syntax.

1. Classify a failure

The PNG file exists, but a raw browser cannot run its import. Which system is responsible: preparation, resource readiness, or movement rules?

2. Follow a resource

Explain the difference between snakeSrc, snakeHead, and context.drawImage(snakeHead, x, y). Where does each belong?

3. Predict a board

The window is 1,025 pixels wide and SNAKE_SIZE is 64. What is the canvas width? How many columns fit? What is the last legal x?

4. Trace movement

The ordered snake coordinates are [(192,64), (128,64), (64,64)]. It moves right without eating. What array results? Which old object becomes the head?

5. Explain a design choice

Why is the Apple instance created after the Snake instance? Why does the loop wait for an image event?

6. Diagnose a random edge

The board width is 640. Why can the current apple algorithm choose x = 640? What should the largest legal x be?

Check your reasoning

1. Preparation/connection: Vite transforms the asset import into something the browser can use. File existence alone is not enough.

2. snakeSrc is a URL string; snakeHead is an image element that loads it; drawImage paints the loaded image into the canvas.

3. 1025 % 64 = 1, so width = 1024, columns = 16, and last x = 960.

4. [(256,64), (192,64), (128,64)]. The old tail object at (64,64) is removed, changed to (256,64), and inserted at the front.

5. Apple.move() reads snake.segments, so snake must exist first. Image loading happens asynchronously; drawing needs a ready image.

6. The helper includes its maximum, and 640 % 64 = 0. The last valid x is 576. Randomize a column index from 0 through 9 instead.

A compact map for your next read-through

When a detail feels disconnected, name the job it performs and trace its inputs and outputs.

Concept	Question it answers	Project anchor
Module / import	What does this file depend on?	main.js lines 1-7
URL / element / loaded data	Where is it, what loads it, when is it usable?	lines 170-179
Class / instance / this	Who owns this state and behavior?	Snake and Apple
Getter / static method	Derived property or class-level helper?	head / getRandomNumber
Array / object reference	What is ordered; what is mutated?	segments; pop/unshift/push
% / cell size	How are positions aligned?	sizeCanvas; Apple.move
Callback / event / timer	What runs later; what triggers it?	onload; keydown; setTimeout
Predicate / some / !	Does any segment occupy this cell?	Apple.move
Update / render	What changes state; what displays it?	move/grow versus draw
npm script / Vite build	How is source prepared and served?	package.json scripts

Primary sources for this guide

Project root: `C:\xampp\htdocs\Class\class\js\74\usingVite`. The guide uses index.html, all of src/main.js and src/style.css, package.json, package-lock.json, .gitignore, and the asset inventory. Adjacent js/74/snake.js and snake.html establish what changed in the Vite migration. Line references refer to the files as reviewed on November 10, 2026.

Official explanations to revisit

- Vite: Getting Started - development server and build workflow.
- Vite: Static Asset Handling - imported URLs and the public directory.
- Vite: Features - CSS imports and HMR.
- MDN: JavaScript Modules - module loading and scope.
- MDN: HTMLMediaElement.play() - playback promises and rejection.
- npm: package.json - scripts, dependencies, and package metadata.

> **Your one-sentence explanation**
>
> Vite prepares and connects the files; the browser loads the resources; input and rules update game state; canvas and audio express that state; a timer repeats the cycle.